

Lección 15: Sentry Errors & Release Health

Manejo Inteligente de Errores en Producción

Agenda

- El Problema de la “Caja Negra”
 - Los 4 Pilares del Error Management
 - Error Classification
 - Release Health
 - User Journey Tracking
 - User Feedback Integration
-

El Problema

Desarrollo vs Producción:

Desarrollo Local

=====

- 1 usuario
- Red perfecta
- Navegador actualizado
- Datos controlados
- Sin interrupciones

Producción

=====

- Miles simultáneos
- Conexiones 3G
- IE11, móviles antiguos
- Datos reales, caracteres especiales
- Usuarios cerrando a medias

Resultado: “Funciona en mi máquina” ≠ “Funciona para todos”

Los 4 Pilares

1. **Clasificación de Errores:** No todos los errores son iguales
 2. **Salud de Releases:** ¿Mi nueva versión rompe algo?
 3. **Seguimiento del Recorrido del Usuario:** ¿Qué hizo el usuario antes?
 4. **Retroalimentación del Usuario:** Deja que usuarios ayuden
-

Clasificación de Errores

Sin clasificación:

"NetworkError: Failed to fetch"

→ ¿Problema del servidor? ¿Red del usuario?

"TypeError: Cannot read property 'x' of undefined"

→ ¿Bug de código? ¿Datos malformados?

Con clasificación:

```
{
  category: 'business-logic',
  severity: 'high',
  probable_cause: 'Bug en cálculo de descuento',
  action_required: 'Arreglar inmediatamente – afecta revenue'
}
```

Tipos de Errores (1/2)

1. Errores de Red:

- Problemas de conectividad del usuario
- No requieren arreglo de código
- Mejora UX (estados de carga, reintentos)

2. Errores de Lógica de Negocio:

- Bugs en tu lógica
 - **Críticos:** Arreglo inmediato
 - Afectan funcionalidad core
-

Tipos de Errores (2/2)

3. Errores de Carga de Chunks:

- Problemas de caché/deployment
- Comunes en SPAs
- Estrategias de cache busting

4. Errores de Terceros:

- Problemas con librerías externas
 - Evaluar actualización o cambio
-

Salud de Releases

Antes de Release Health:

1. Haces deploy
2. Esperas quejas de usuarios
3. Intentas conectar errores con deployment

Con Release Health:

1. Haces deploy
 2. Ves: "Versión 2.1.0 tiene 300% más errores"
 3. Rollback informado en minutos
-

Dashboard de Salud de Release

Release v2.1.0 (deployed hace 10 min)

```
=====
Tasa de Crash: 2.1% (↑ 150% desde v2.0.9)
Errores/hora: 450 (↑ 300% desde anterior)
Usuarios afectados: 1,240 usuarios únicos
Tiempo de detección: 3 minutos
Estado: 🚨 DEGRADADO – Considerar rollback
```

Métricas Clave:

- Tasa de Crash: % sesiones que terminan en error
- Tasa de Errores: Errores por minuto
- Impacto de Usuario: Usuarios únicos afectados
- Tiempo de Recuperación: Cuánto tardas en detectar/solucionar

Recorrido del Usuario: Breadcrumbs

Sin breadcrumbs:

```
Error: Cannot process payment
Stack trace: payment.js:127
```

¿Qué información útil tienes? Casi ninguna.

Con breadcrumbs:

```
Recorrido del usuario antes del error:
1. 14:32:01 – Click en "Add to Cart" (laptop-pro)
2. 14:32:03 – Carrito actualizado (items: 1, total: $1200)
3. 14:32:05 – Click en "Apply Discount" (SAVE10)
4. 14:32:07 – Descuento aplicado (10% off, total: $1080)
5. 14:32:10 – Click en "Checkout"
6. 14:32:18 – Click en "Pay Now"
7. 14:32:19 – ERROR: Cannot process payment
```

Ahora puedes reproducir exactamente el problema.

Tipos de Breadcrumbs (1/2)

Interacciones de UI:

- Clicks, envíos de formularios
- Presiones de botones, swipes

Navegación:

- Cambios de página
- Transiciones de rutas

Peticiones de red:

- Llamadas a API
 - Respuestas
-

Tipos de Breadcrumbs (2/2)

Eventos de negocio:

- Descuento aplicado
- Item agregado al carrito
- Checkout iniciado

Performance:

- Operaciones lentas
 - Advertencias de memoria
-

Integración de Retroalimentación del Usuario

El problema:

- Usuarios experimentan errores pero no reportan
- Cuando reportan: email/redes sin contexto técnico

Con retroalimentación integrada:

1. Error sucede

2. Modal: "¿Algo no funcionó bien?"
 3. Usuario añade contexto: "No me dejaba pagar"
 4. Se captura automáticamente:
 - Screenshot
 - Estado de la app
 - Pasos previos
 5. Reporte llega con contexto completo
-

Beneficios de la Retroalimentación

Contexto humano:

- Usuarios explican QUÉ intentaban hacer

Captura inmediata:

- Screenshot en el momento exacto

Menos falsos positivos:

- Usuarios distinguen bugs de confusión

Priorización:

- Si usuarios reportan, realmente importa
-

Error Fingerprinting

Sin fingerprinting:

```
TypeError: Cannot read 'name' of undefined (user: 123)
TypeError: Cannot read 'name' of undefined (user: 456)
TypeError: Cannot read 'name' of undefined (user: 789)
```

→ 100 alertas del mismo error

Con fingerprinting:

```
fingerprint = [
```

```
'TypeError',  
'Cannot read property name',  
'user-profile-component'  
]
```

→ 1 alerta: “TypeError en user-profile (100 usuarios)”

Source Maps

¿Por qué son críticos?

Código original:

```
function calculateDiscount(cart) {  
  if (cart.items.length >= 5) {  
    return cart.total * 0.1  
  }  
}
```

Código en producción (minificado):

```
function a(b) {  
  return b.c.length >= 5 ? b.d * 0.1 : 0  
}
```

Source map conecta:

```
Error línea 1, columna 25 (minificado)  
→ Error línea 3, columna 12 de calculateDiscount()
```

Session Tracking

Contexto de sesión:

```
{  
  sessionId: "session-123-abc",  
}
```

```
duration: 1847000, // ms desde inicio
userAgent: "Chrome/120.0 on MacOS",
viewport: "1920x1080",
language: "es-ES",
connectionType: "4g"
}
```

¿Por qué es útil?

- Patrones temporales: “Errores después de 30 min” (memory leak?)
- Correlaciones: “Safari tiene más errores de chunks”
- Reproducción: Recrear entorno exacto
- Segmentación: “Solo móvil + 3G”

Métricas que Importan

Métricas Vanidosas:

- Total errores capturados
- Número usuarios monitoreados
- Cantidad breadcrumbs

Métricas Accionables:

- **Tiempo de Detección:** ¿Cuánto tardas en saber?
- **Tiempo de Resolución:** ¿Cuánto en arreglar?
- **Tasa de Impacto al Usuario:** % usuarios con errores
- **Detección de Regresiones:** % deployments con nuevos errores

¿Qué es un KPI?

KPI (Key Performance Indicator): Indicador Clave de Rendimiento

Definición:

- Métrica cuantificable que mide el éxito de objetivos específicos
- Responde: “¿Estamos logrando nuestros objetivos?”

Ejemplo práctico:

Objetivo: Mantener app estable en producción

KPI: Error Budget = 0.1% error rate máximo/semana

- Si error rate < 0.1% →  Objetivo cumplido
- Si error rate > 0.1% →  Acción requerida

En Release Health: KPIs miden salud de cada deployment

KPIs de Release Health

1. **Error Budget:** “Esta semana: 0.1% error rate máximo”
 2. **Éxito de Deployment:** “95% deployments sin regresiones”
 3. **MTTR:** “Promedio 15 min desde detección hasta fix”
 4. **Impacto al Cliente:** “Errores en checkout peso 10x”
-

Ejercicio 1: Error Boundary con Degradación Elegante

Prompt:

Actúa como un frontend engineer implementando manejo elegante de errores co

CONTEXT0: React Error Boundaries capturan errores en componentes hijos (pre white screen of death – pantalla blanca completa). Sin ErrorBoundary: error 1 componente rompe toda la app. Con ErrorBoundary: error aislado + fallback mostrado + app sigue funcionando. Sentry.ErrorBoundary: combina React error boundary + Sentry reporting automático. showDialog prop: muestra diálogo pa que usuario reporte problema (retroalimentación del usuario). Graceful degr (degradación elegante): app degrada elegantemente en vez de romperse comple Component stack: árbol de componentes donde ocurrió error (útil para debugg

TAREA: Envuelve ShoppingCart con Sentry.ErrorBoundary para error handling g

IMPLEMENTACIÓN:

- Archivo: src/App.tsx (o donde ShoppingCart se renderiza)
- Import: import * as Sentry from '@sentry/react'

ERROR BOUNDARY WRAPPER:

```
<Sentry.ErrorBoundary
  fallback={<div>Something went wrong with the shopping cart</div>}
  showDialog
>
  <ShoppingCart />
</Sentry.ErrorBoundary>
```

PROPS:

- fallback: UI que se muestra cuando hay error (en vez de crash)
- showDialog: muestra feedback dialog para que usuario reporte issue

TESTING ERROR BOUNDARY:

- Archivo: src/features/shopping-cart/components/CartItem.tsx
- Agregar error intencional:

```
if (item.id === 'test-error') {
  throw new Error('Test error boundary - intentional')
}
```

VALIDACIÓN EN DASHBOARD DE SENTRY:

1. Agregar item con id 'test-error' al carrito
2. App debe mostrar fallback UI (NO crash completo)
3. Ir a dashboard de Sentry → Issues
4. Verificar:
 - Título del error: "Test error boundary - intentional"
 - Component stack visible (muestra jerarquía)
 - Diálogo de retroalimentación apareció (si usuario lo completó)

VALIDACIÓN: Error capturado + fallback UI mostrado + app continúa funcionan

Aprende: Error Boundaries previenen crashes completos

y mejoran UX en producción

Ejercicio 2: Breadcrumbs Personalizados para Recorrido del Usuario

Prompt:

Actúa como un engineer agregando telemetría del recorrido del usuario con S

CONTEXTO: Breadcrumbs = rastro de eventos antes de error (responde "**¿QUÉ hi usuario antes del crash?**"). Sin breadcrumbs: error aislado sin contexto. Con breadcrumbs: timeline completo (navegación → clicks → llamadas API → error) Sentry.addBreadcrumb() agrega eventos custom al rastro. Niveles de breadcrumb (normal), warning (sospechoso), error (problemático). Category: agrupa eventos relacionados (**'navigation', 'cart', 'api'**). Seguimiento del Recorrido del U reconstruir pasos exactos del usuario para reproducir bugs. Breadcrumbs automaticamente Sentry captura clicks/navegación automáticamente, breadcrumbs custom agrega de negocio.

TAREA: Agrega breadcrumb custom cuando usuario agrega item al carrito.

UBICACIÓN:

- Archivo: src/features/product-catalog/components/ProductCard.tsx
- Handler: handleAddToCart (o similar)

IMPLEMENTACIÓN:

- Import: import * as Sentry from **'@sentry/react'**
- Ubicación: ANTES de llamar addToCart() (capturar intención)

BREADCRUMB STRUCTURE:

```
Sentry.addBreadcrumb({
  message: 'Added item to cart',
  category: 'cart',
  level: 'info',
  data: {
    productId: product.id,
    productName: product.name,
    price: product.price,
    quantity: quantity
  }
})
```

PROPS REQUERIDAS:

- message: Descripción human-readable de la acción
- category: Tipo de evento (cart, navigation, api, etc.)
- level: **'info'** (acción normal) | **'warning'** | **'error'**
- data: Contexto adicional (IDs, valores relevantes)

TESTING BREADCRUMBS:

1. Agregar item al carrito (ejecuta breadcrumb)
2. Generar error intencional después (throw new Error() en siguiente acción)
3. Ir a dashboard de Sentry → Issue → tab Breadcrumbs
4. Verificar timeline:
 - Navigation: Visited /products (automático)
 - Acción del usuario: Added item to cart (breadcrumb custom)
 - Error: [tu error intencional]

VALIDACIÓN: Breadcrumb debe aparecer en timeline ANTES del error con data c

Aprende: Breadcrumbs custom enriquecen contexto del recorrido

del usuario para debugging efectivo

Puntos Clave

1. **Clasificación:** Categoriza errores automáticamente
2. **Salud de Releases:** Detecta regresiones en minutos
3. **Breadcrumbs:** Captura recorrido completo del usuario
4. **Retroalimentación del Usuario:** Contexto humanizado + técnico
5. **Fingerprinting:** Agrupa errores similares
6. **Source Maps:** Debug con código original
7. **Contexto de Sesión:** Reproduce entorno exacto